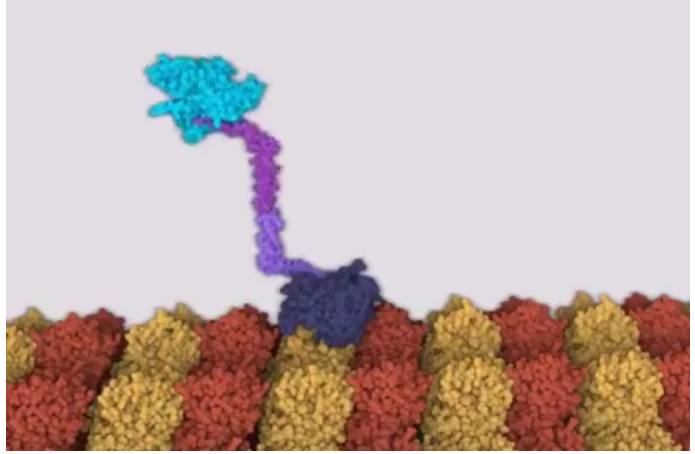


VU Computer Animation (186.139)

Lecture 2: Interpolation-Based Animation

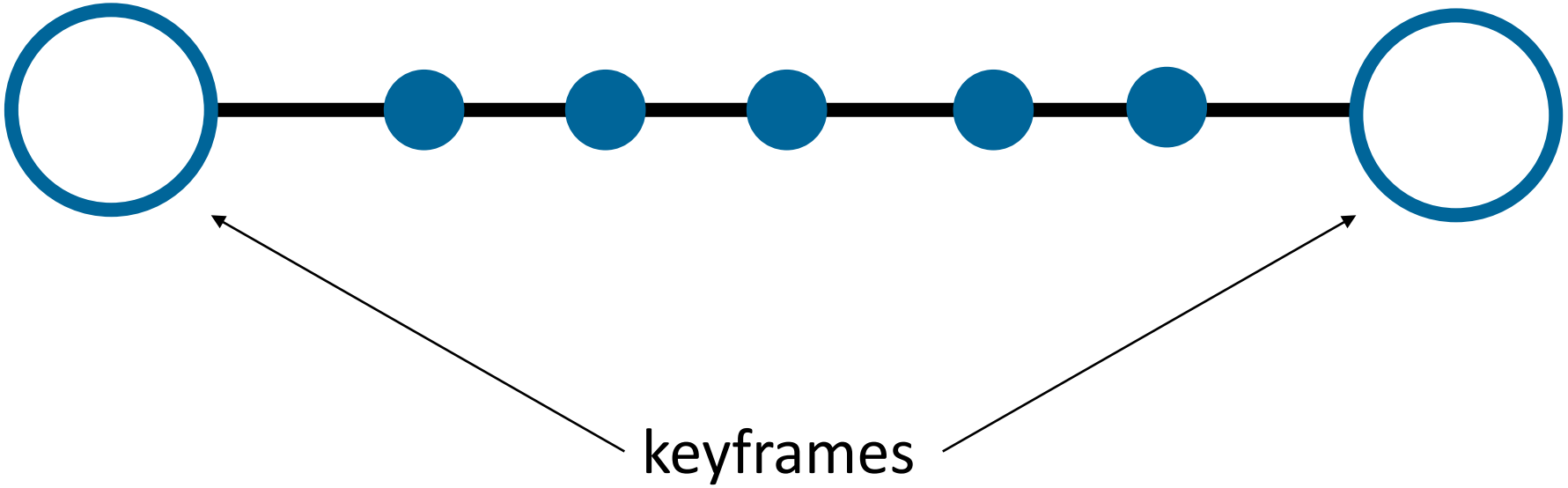
Renata Raidou

Institute of Visual Computing and Human-Centered Technology, TU Wien, Austria

Part A: Interpolating Values

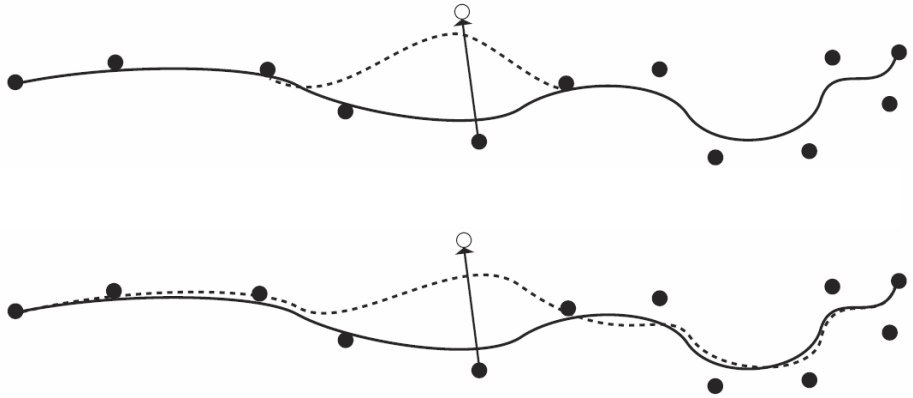
Animation is All About Interpolating Values

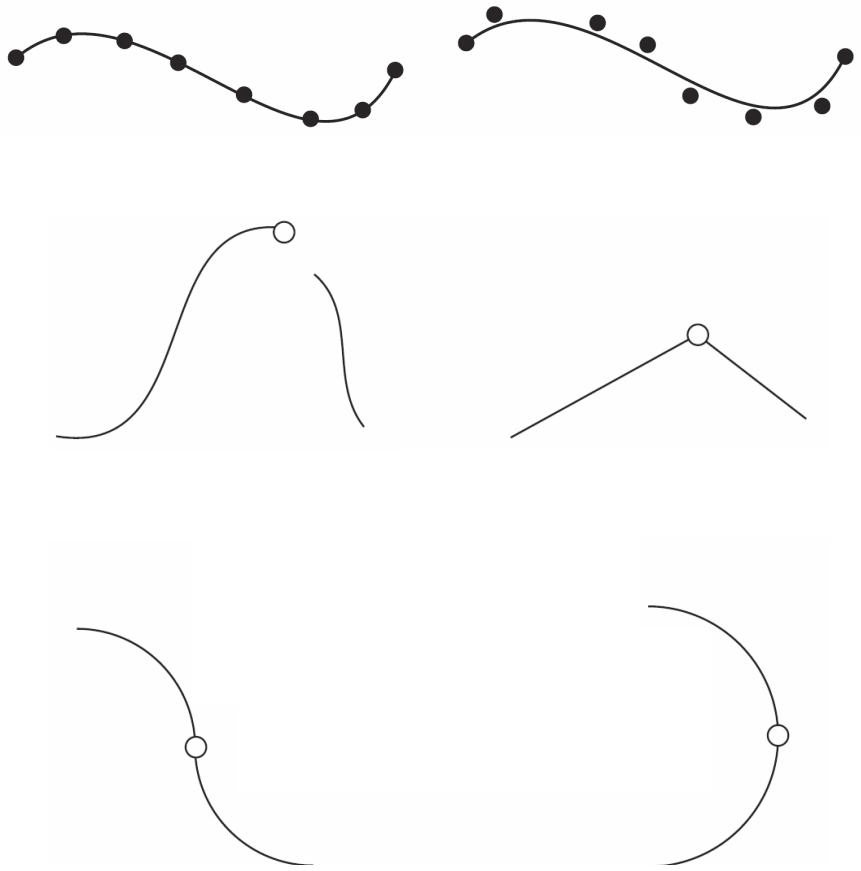
- Hint: Observe the behavior from nature



Interpolation Paths

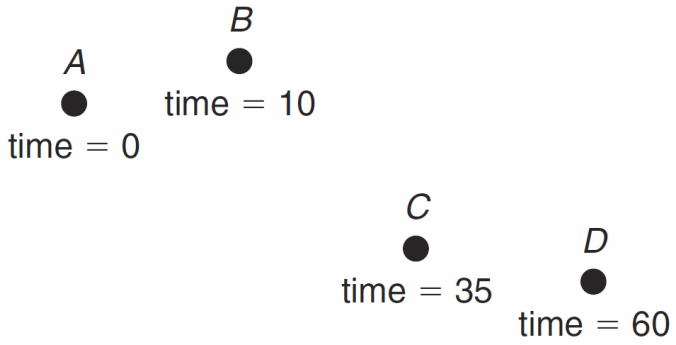
- Interpolation reconstructs a continuum between discrete sample values
- Parametric curves / paths
  - Interpolative vs. approximative
  - Complexity
  - Continuity C0, C1, C2
  - Global and local control

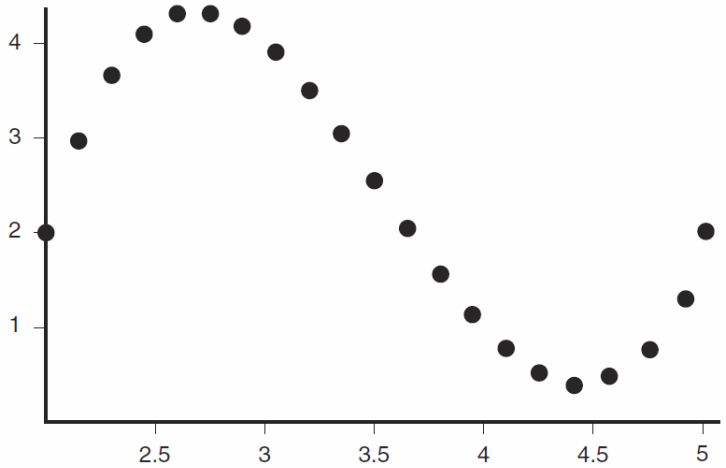




Motion of a Point along a Curve

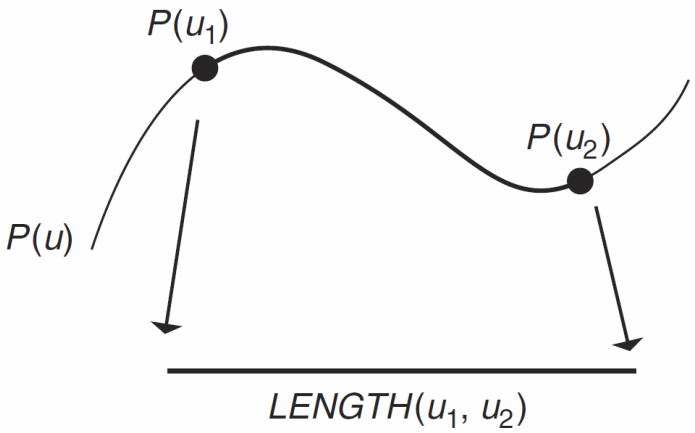
- Camera movement or object movement (e.g. limb animation)
- Parametric curve $\boldsymbol{P}(u)$





1. Arc length parameterization
   s = S(u), u = U(s)
2. Speed control
   s = ease(t)

$$
\begin{aligned}
\boldsymbol{P}(u) &= (x(u),\ y(u),\ z(u)) \\
x(u) &= a_x u^3 + b_x u^2 + c_x u + d_x \\
y(u) &= a_y u^3 + b_y u^2 + c_y u + d_y \\
z(u) &= a_z u^3 + b_z u^2 + c_z u + d_z
\end{aligned}
$$



Arc Length Computation: Analytic Approach

- Length of a curve expressed via integral

$$s = \int_{u_a}^{u_b} |d\boldsymbol{P}/du|\,du$$

$$d\boldsymbol{P}/du = ((dx(u)/(du)), (dy(u)/(du)), (dz(u)/(du)))$$

$$|d\boldsymbol{P}/du| = \sqrt{(dx(u)/du)^2 + (dy(u)/du)^2 + (dz(u)/du)^2}$$

Ex. $P(u) = au^3 + bu^2 + cu + d$

$$dx(u)/du = 3a_x u^2 + 2b_x u + c_x$$

$$dy(u)/(du)$$

$$dz(u)/(du)$$

$$Au^4 + Bu^3 + Cu^2 + Du + E$$

$$A = 9(a_x^2 + a_y^2)$$
$$B = 12(a_x b_x + a_y b_y)$$
$$C = 6(a_x c_x + a_y c_y) + 4(b_x^2 + b_y^2)$$
$$D = 4(b_x c_x + b_y c_y)$$
$$E = c_x^2 + c_y^2$$

Arc Length Parameterization: Forward Differencing

- Sample the curve with small *u* steps
- Approximate the arc length by sum of Euclidean distances between samples
- For arbitrary value of *u*, obtain *s* by linear interpolation

$$s = G[i] + \frac{u - V[i]}{V[i+1] - V[i]}(G[i+1] - G[i])$$

- For arbitrary *s* obtain *u* by analogous function
- For a parameter interval (u_a, u_b), $s = s_b - s_a$

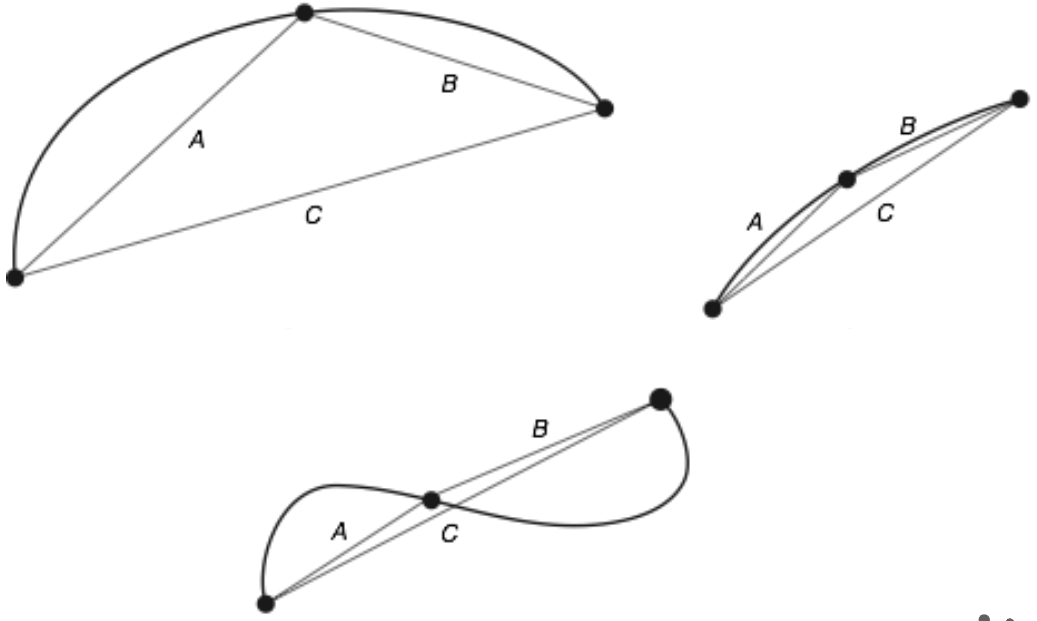
i	*u (V[i])*	*s (G[i])*
0	*0.00*	*0.000*
1	*0.05*	*0.080*
2	*0.10*	*0.150*
3	*0.15*	*0.230*
…	*…*	*…*
20	*1.00*	*1.000*

Arc Length Parameterization: Adaptive Approach

- Sample the curve adaptively instead of uniform sampling
- Perform backtracking to compute arc length table

$$\left| ||P(0.0) - P(1.0)|| - \left(||P(0.0) - P(0.5)|| + ||P(0.5) - P(1.0)|| \right) \right| < \varepsilon$$

- For each subdivision *n* compute $\varepsilon/2^n$
- Start with non-adaptive subdivision and continue with an adaptive scheme to avoid erroneously passing the threshold condition



Arc Length Parameterization: Gaussian Quadrature

- Instead of approximating the function by linear segments, the Gaussian quadrature approximates the function by higher order functions $g(t) = At^4 + Bt^3 + Ct^2 + D$

$$\int_{-1}^{1} f(u) = \sum_i w_i f(u_i)$$

$$t = f(u) = \frac{(b-a)}{2}u + \frac{b+a}{2}$$

$$\int_a^b g(t)dt = \int_{-1}^{+1} g(f(u))f'(u)du$$

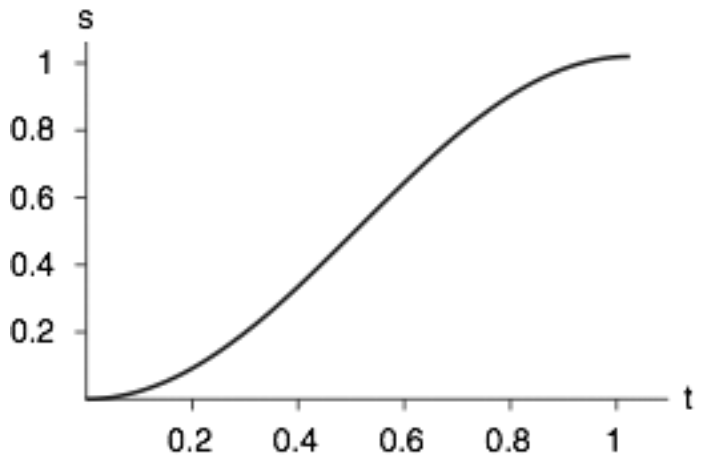
$$= \left(\frac{b-a}{2}\right)\int_{-1}^{+1} g\left(\frac{(b-a)}{2}u + \frac{b+a}{2}\right)du$$

Number of points, n	Points, x_i	Weights, w_i
1	0	2
2	$\pm\sqrt{\frac{1}{3}}$	1
3	0	$\frac{8}{9}$
	$\pm\sqrt{\frac{3}{5}}$	$\frac{5}{9}$
4	$\pm\sqrt{\frac{3}{7} - \frac{2}{7}\sqrt{\frac{6}{5}}}$	$\frac{18+\sqrt{30}}{36}$
	$\pm\sqrt{\frac{3}{7} + \frac{2}{7}\sqrt{\frac{6}{5}}}$	$\frac{18-\sqrt{30}}{36}$
5	0	$\frac{128}{225}$
	$\pm\frac{1}{3}\sqrt{5 - 2\sqrt{\frac{10}{7}}}$	$\frac{322+13\sqrt{70}}{900}$
	$\pm\frac{1}{3}\sqrt{5 + 2\sqrt{\frac{10}{7}}}$	$\frac{322-13\sqrt{70}}{900}$

Speed Control: Time-Distance Plots

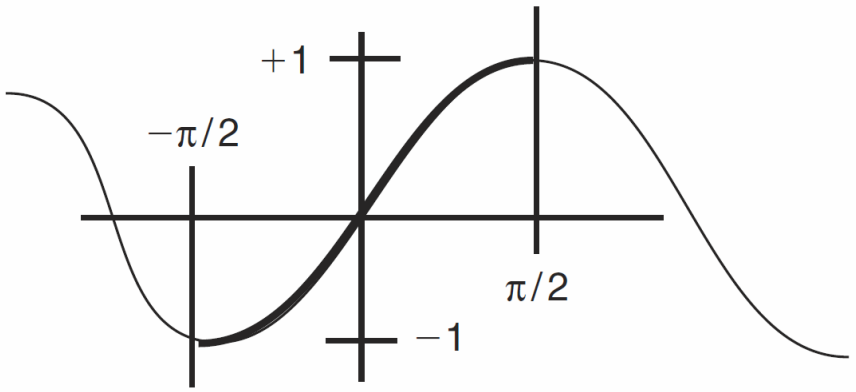
- Time-distance function operates on a normalized arc length
  - Monotonic
  - Continuous

$$\boldsymbol{p} = \boldsymbol{P}(U(S(t)))$$



- Easy-in and Ease-out: *S(t) = (2-t)t*
- Sinusoidal Ease-in and Ease-out

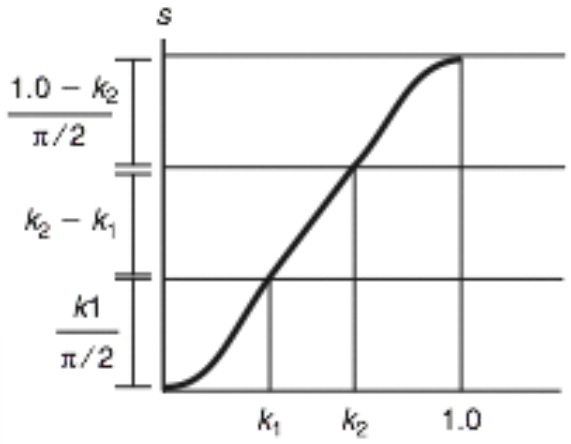
$$s = ease(t) = \frac{\sin\left(t\pi - \frac{\pi}{2}\right) + 1}{2}$$

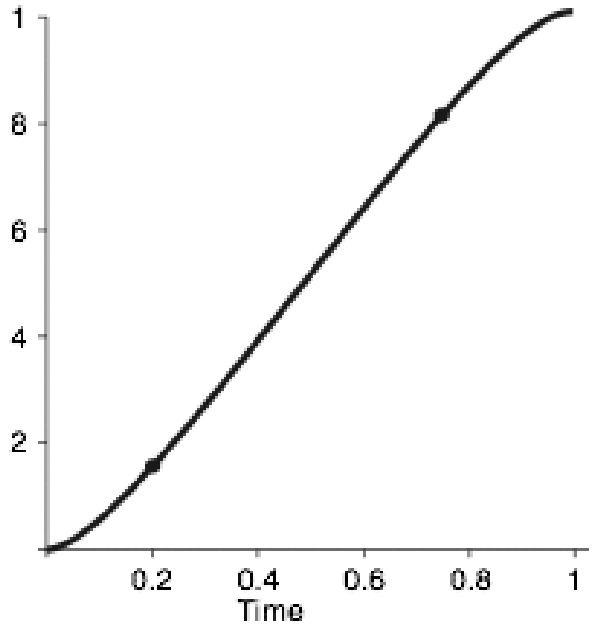


Speed Control: Time-Distance Plots

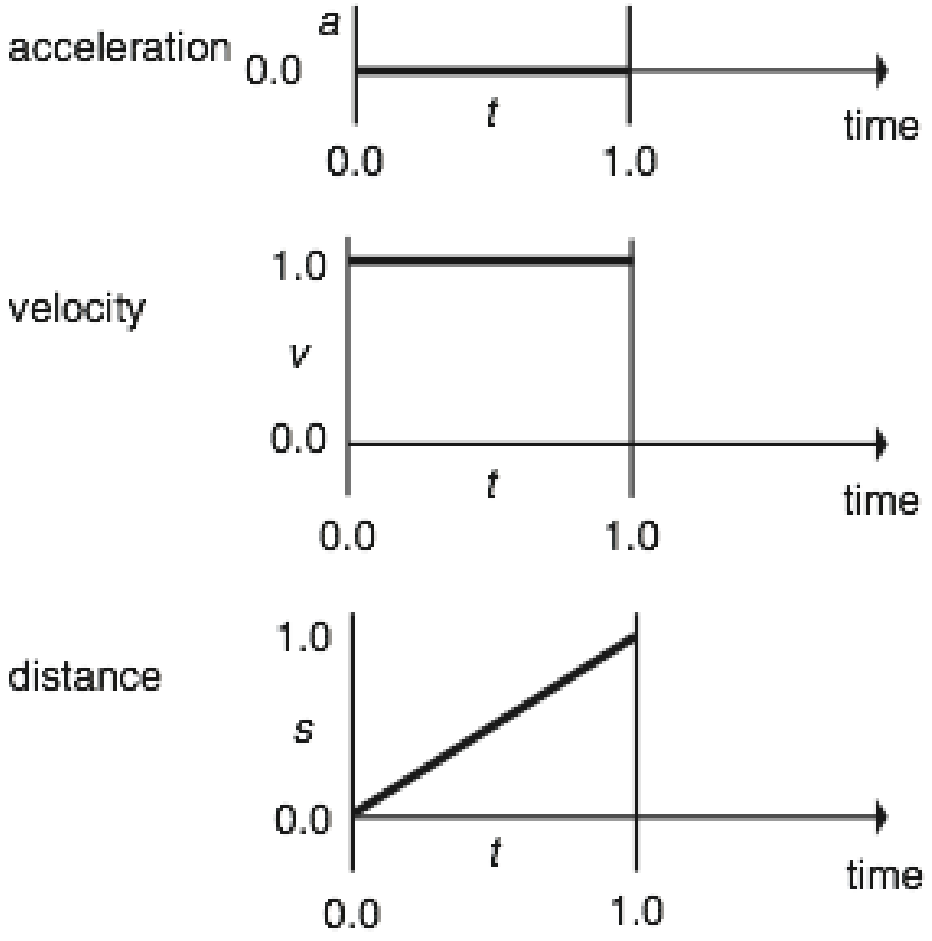
$$ease(t)\begin{cases} = \left(k_1 \frac{2}{\pi}\left(\sin\left(\frac{t}{k_1}\frac{\pi}{2} - \frac{\pi}{2}\right) + 1\right)\right)/f & t \le k_1 \\ = \left(k_1 \frac{2}{\pi} + t - k_1\right)/f & k_1 \le t \le k_2 \\ = \left(k_1 \frac{2}{\pi} + k_2 - k_1 + \left((1-k_2)\frac{2}{\pi}\right)\sin\left(\left(\frac{t-k_2}{1-k_2}\right)\frac{\pi}{2}\right)\right)/f & k_2 \le t \end{cases}$$

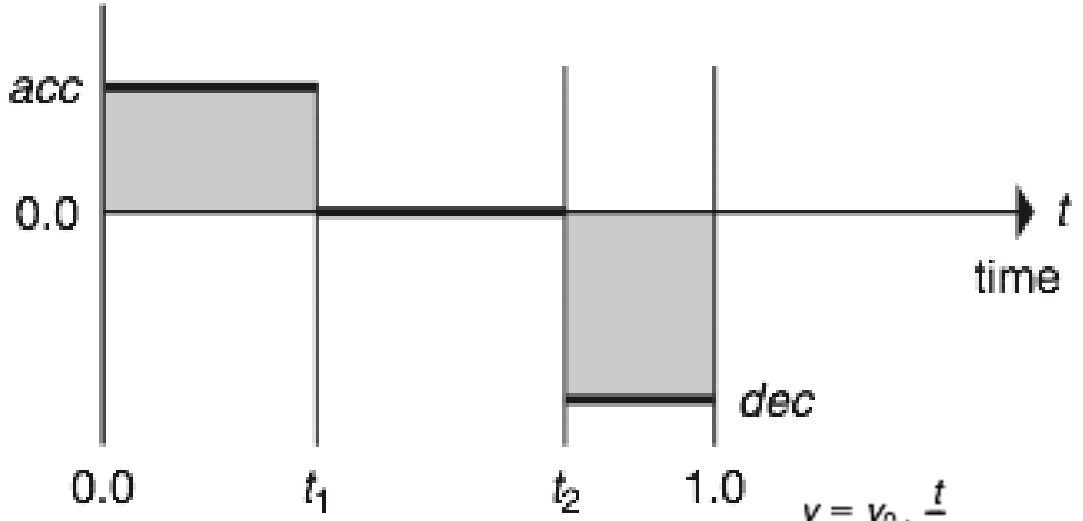
$$f = k_1\ 2/\pi + k_2 - k_1 + (1-k_2)(2/\pi)$$

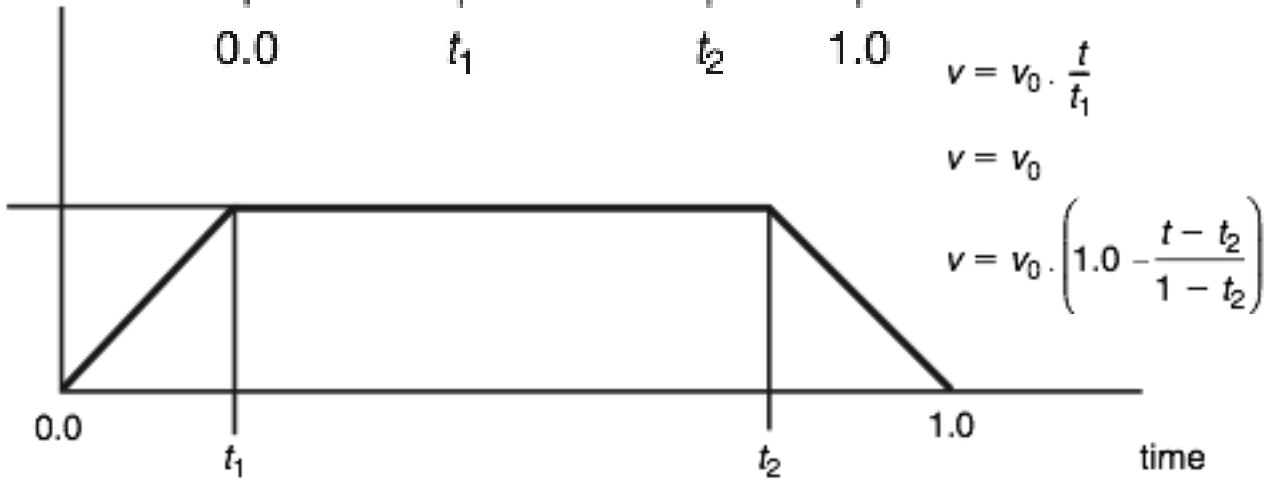




Speed Control: Acceleration, Velocity, Distance Plots







$v = v_0 \cdot \frac{t}{t_1}$ $\quad 0.0 < t < t_1$

$v = v_0$ $\quad t_1 < t < t_2$

$v = v_0 \cdot \left(1.0 - \frac{t - t_2}{1 - t_2}\right)$ $\quad t_2 < t < 1.0$

$$1 = A_{acc} + A_{constant} + A_{dec}$$

$$1 = \frac{1}{2} v_0 t_1 + v_0 (t_2 - t_1) + \frac{1}{2} v_0 (1 - t_2)$$

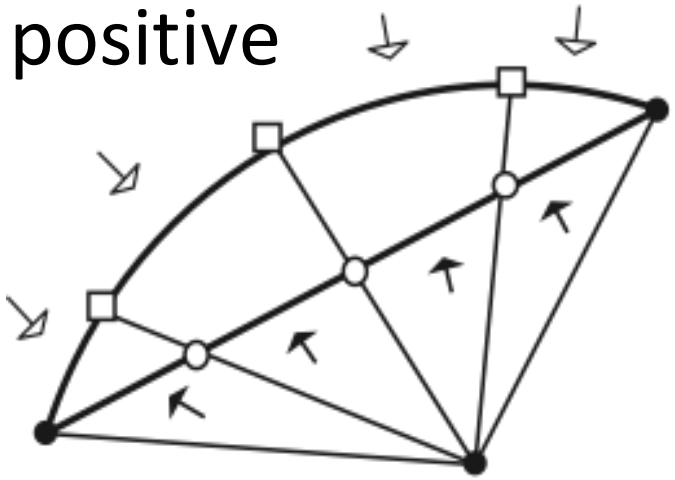
$$v_0 = \frac{2}{(t_2 - t_1 + 1)}$$

Interpolation of Orientations

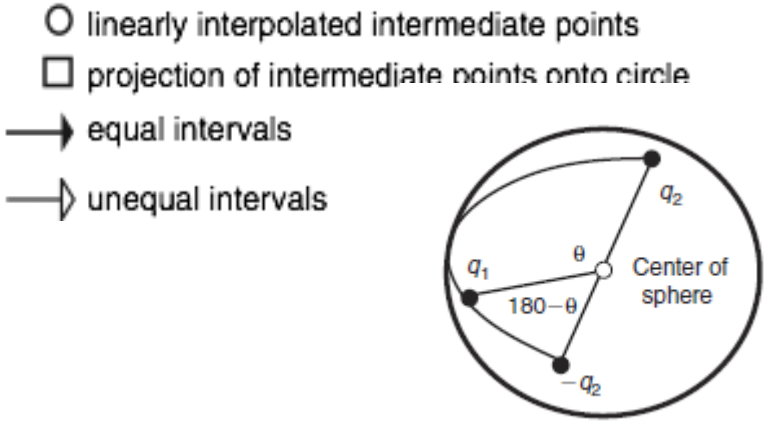
- Component-wise interpolation of quaternions q_1 and q_2 results in a satisfactory rotation, but the speed of rotation varies
- Find out whether cosine between q_1 and is q_2 positive

$$\cos(\theta) = q_1 \cdot q_2 = s_1 s_2 + v_1 \cdot v_2$$

- If negative perform interpolation between q_1 and $-q_2$

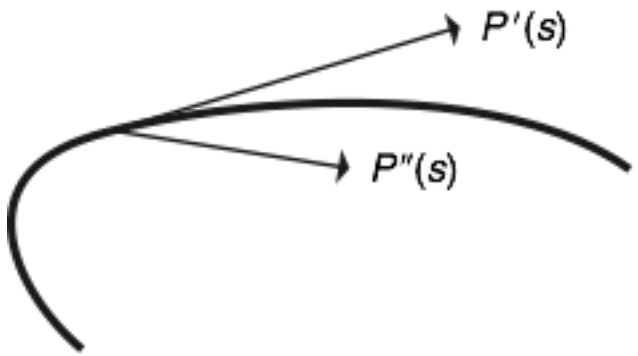


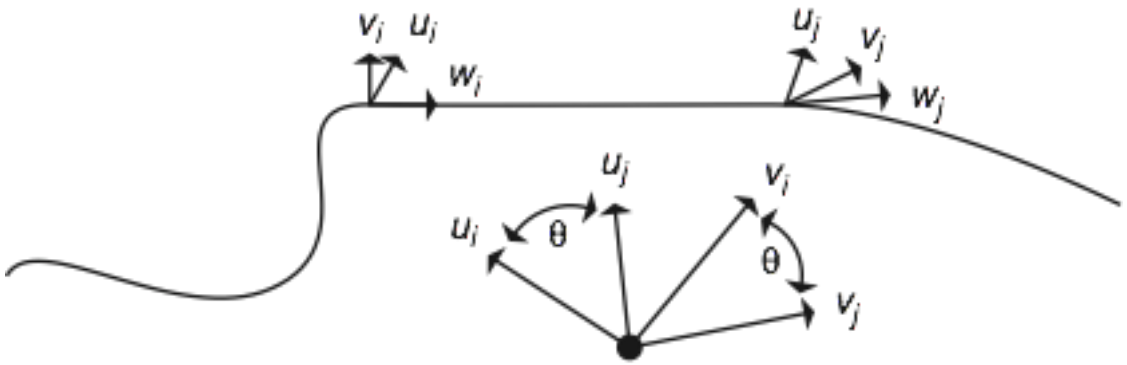
$$slerp(q_1, q_2, u) = \frac{\sin((1-u)\theta)}{sin(\theta)} q_1 + \frac{\sin(u\theta)}{sin(\theta)} q_2$$



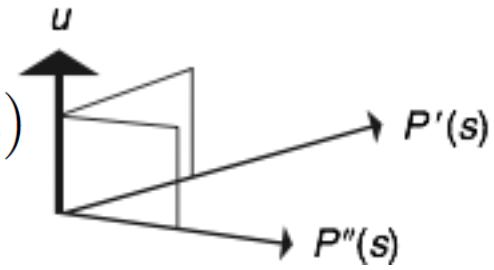
?

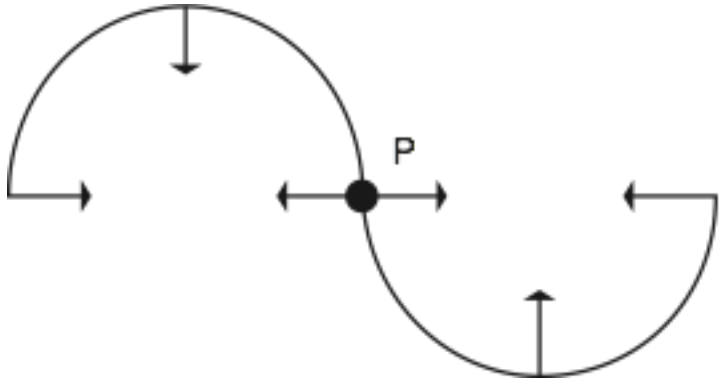
Working with Paths: Frenet Frame

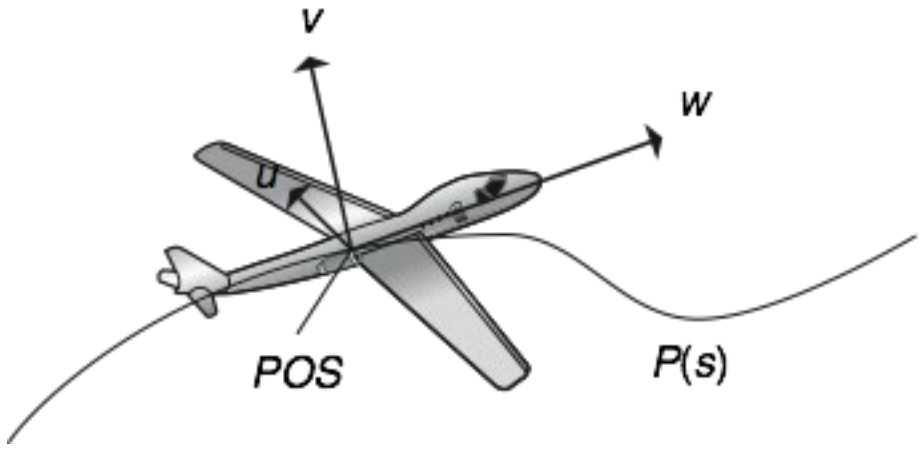




$$\mathrm{w} = \mathrm{P}'(\mathrm{s})$$
$$\mathrm{u} = \mathrm{P}'(\mathrm{s}) \times \mathrm{P}''(\mathrm{s})$$
$$\mathrm{v} = \mathrm{u} \times \mathrm{w}$$







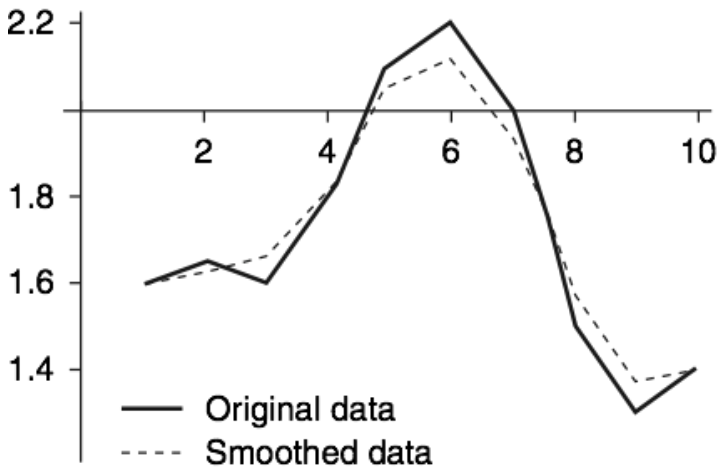
Working with Paths: Center of Interest

- Fixation on an object by selecting its center of gravity as COI
- COI focusing on a path ahead POS: **P**(s); COI: **P**(s+Δs)
- COI focuses on smoothed path
- COI has entirely separate path **C**(s)
- COI is fixated on a point for couple of frames with a smooth transition to another point of fixation

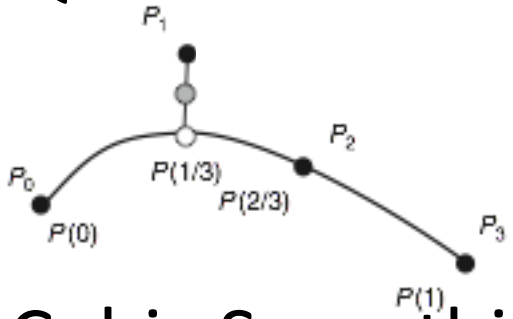
Working with Paths: Polynomial Smoothing

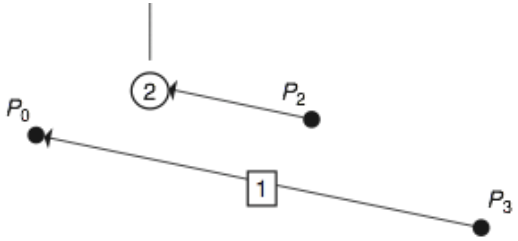
- Linear Smoothing

$$P_i' = \frac{P_i + \frac{P_{i-1}+P_{i+1}}{2}}{2} = \frac{1}{4}P_{i-1} + \frac{1}{2}P_i + \frac{1}{4}P_{i+1}$$

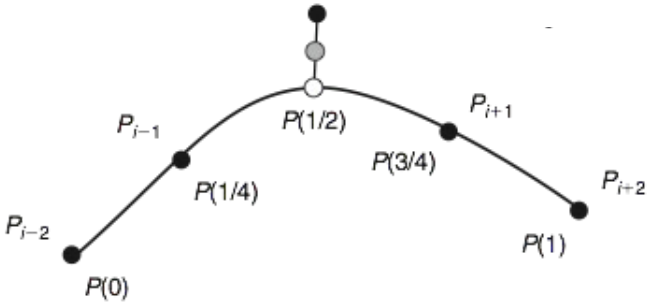


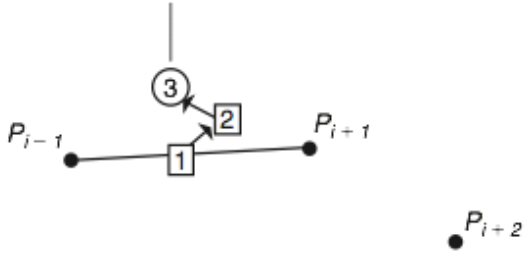
- Quadratic Smoothing

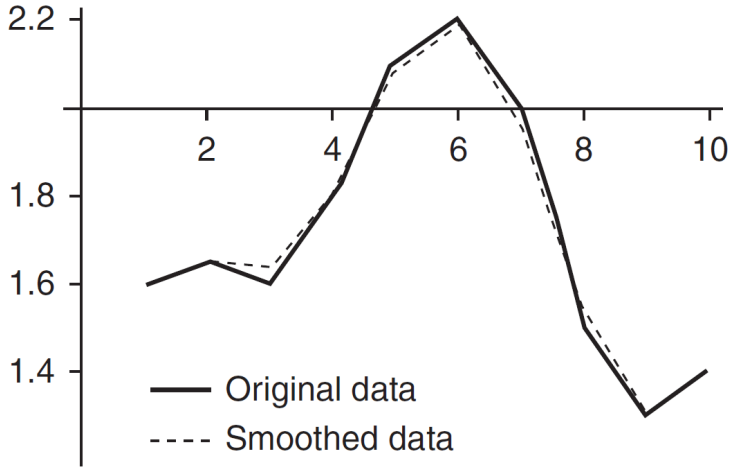




- Cubic Smoothing

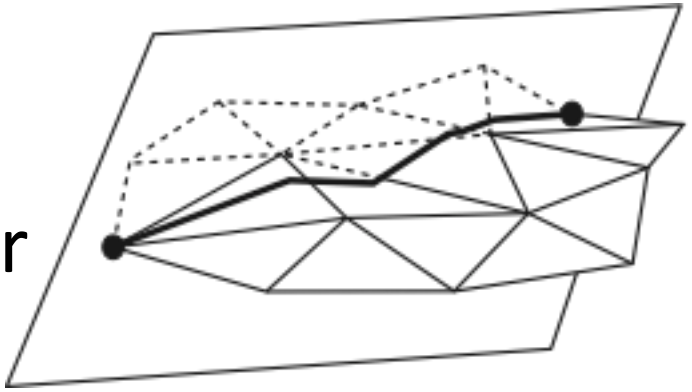






Working with Paths: Determining Path along Surface

- Shortest path along surface can be approximated with intersection between surface and a plane containing the points and generally perpendicular to the surface (contains vector connecting points and another vector is average of two normal at these points)



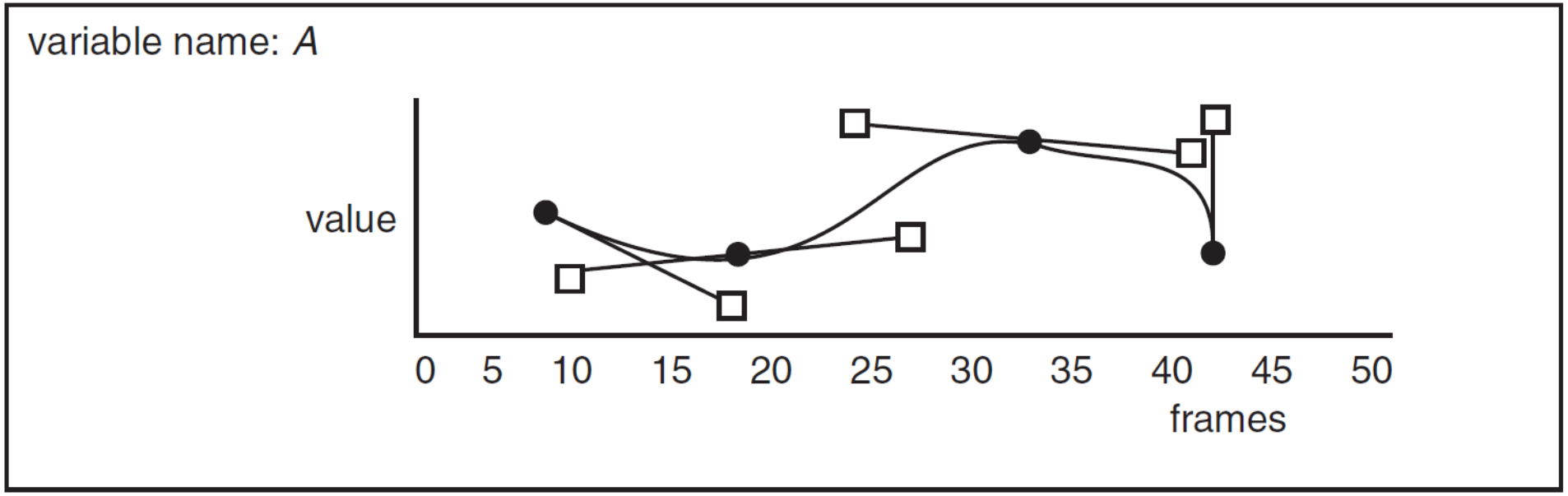
- For parametric surface (u,v) the interpolation between $\mathbf{P}_1\,(u_1, v_1)$ and $\mathbf{P}_2\,(u_2, v_2)$ can go along component-wise linear interpolation of (u,v) coordinates
- Along edges calculate cosine between edges and direction to destination point. Highest cosine value is the edge to proceed.

?

Part B: Interpolation-Based Animation

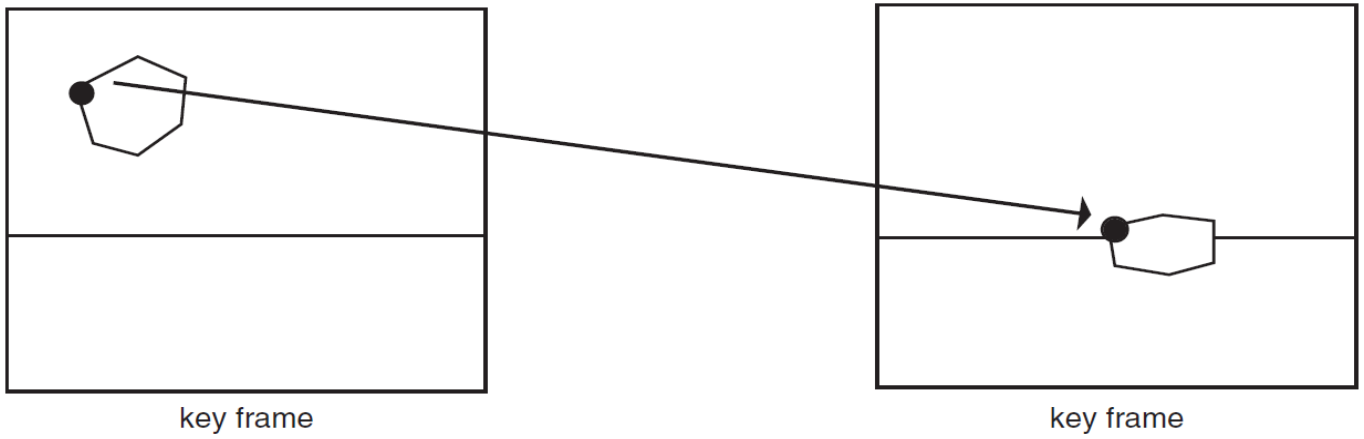
Key-Frame Animation

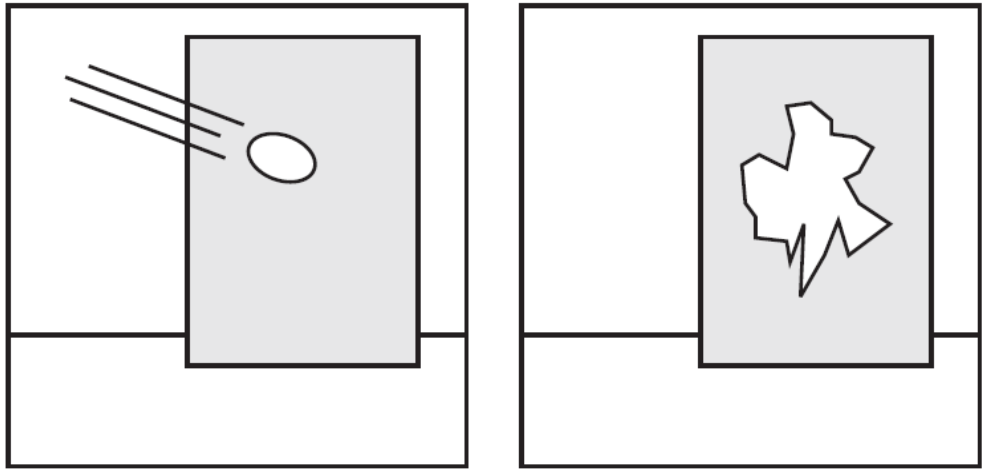
- Key frames in traditional animation were created by master animators, the intermediate frames were filled in by animators in training
- Computer-based animation uses interpolation between key-frames to replace the manual process

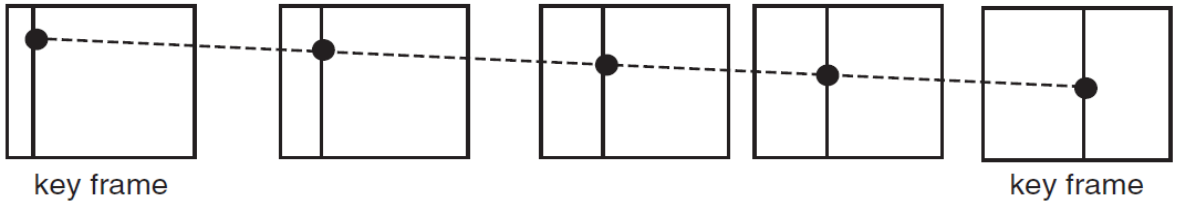


Key-Frame Interpolation

- Simple cases when point correspondence is known can be solved with a simple linear interpolation or higher-order polynomials

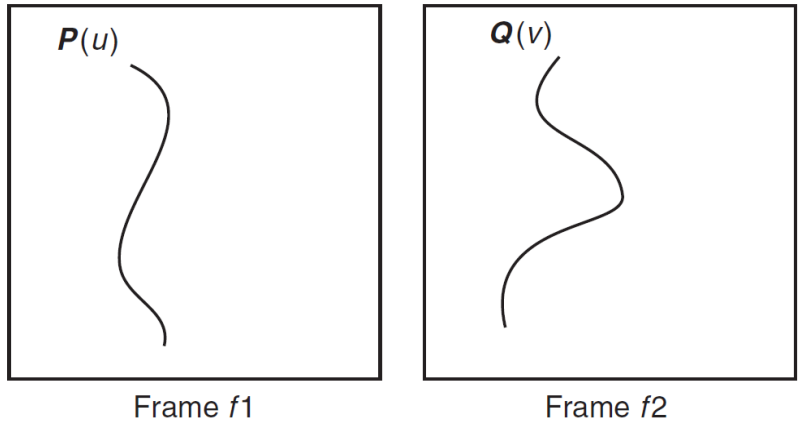


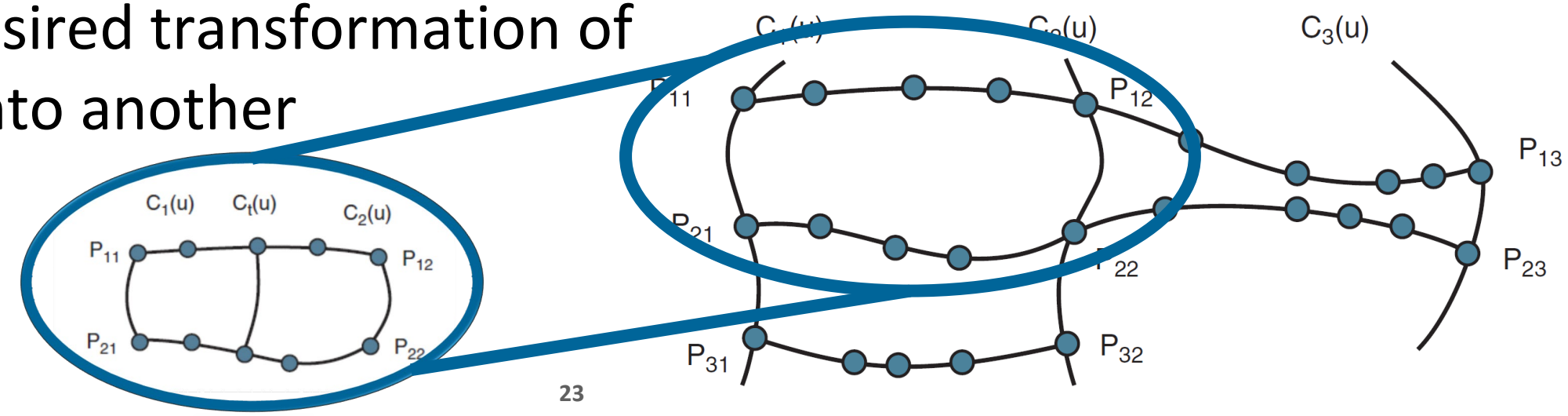




Parametric Curves Interpolation

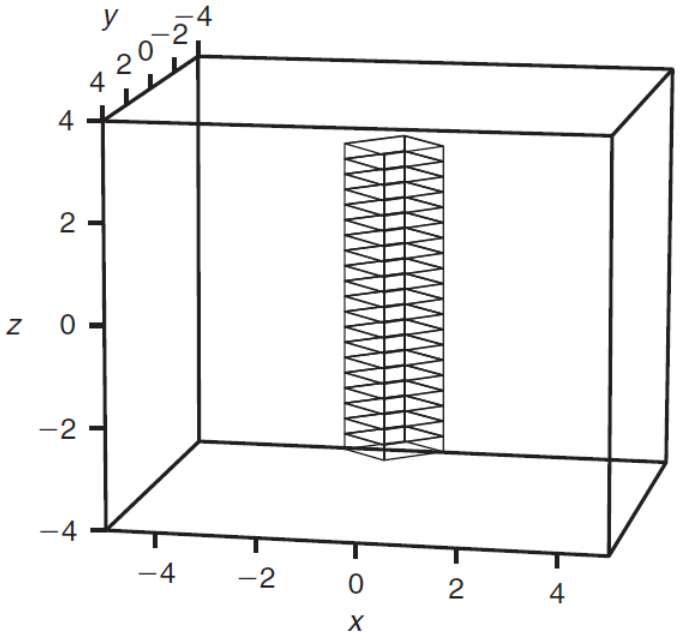
- Interpolate control points can lead to an acceptable solution
- P(0) → Q(0), P(1) → Q(1)
- Interpolate between points
  $(1-\alpha)\, P(u_i) + \alpha\, Q(v_i)$: $\alpha \in [0..1]$
- Moving point constraint:
  Use surface patch parameterization to define desired transformation of one curve into another

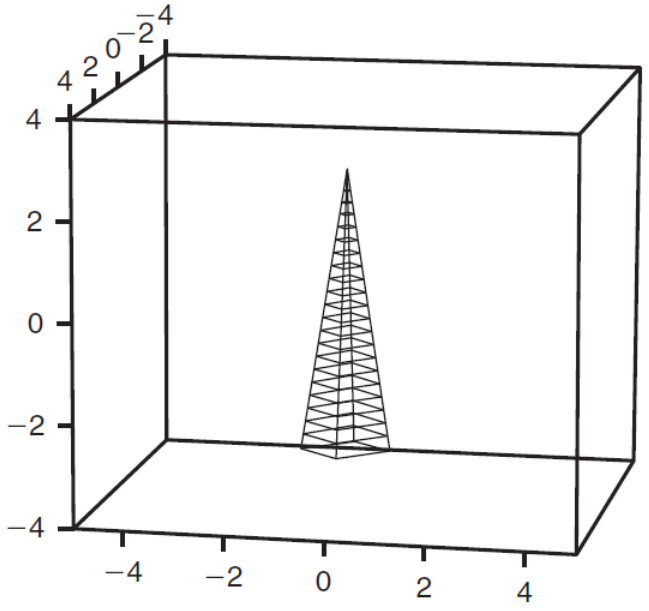


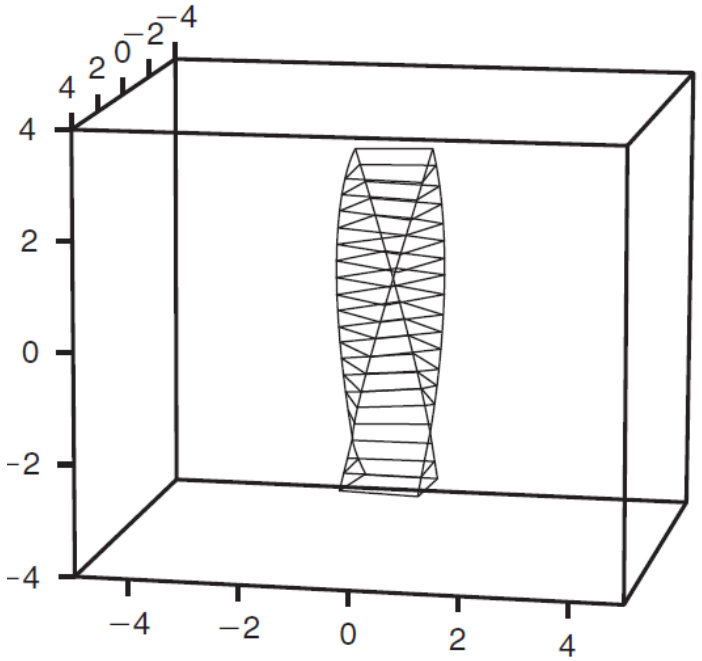


Embedding Space Deformation: Global

- Transformation matrix as a function of point position







$$s(z) = \frac{(\max z - z)}{(\max z - \min z)}$$

$$x' = s(z)x$$
$$y' = s(z)y$$
$$z' = z$$

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} s(z) & 0 & 0 \\ 0 & s(z) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$P' = M(p)p$$

k = twist factor

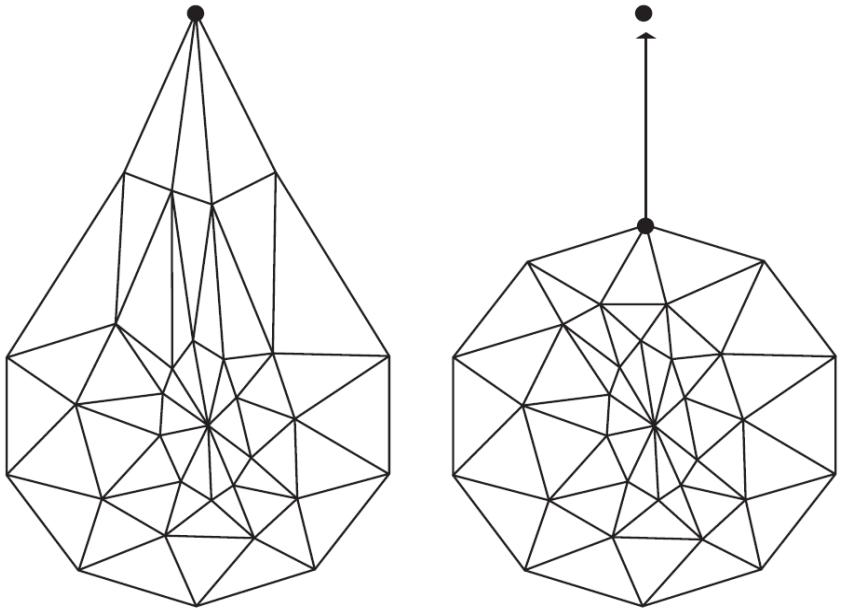
$$x' = x\cos(kz) - y\sin(kz)$$
$$y' = x\sin(kz) + y\cos(kz)$$
$$z' = z$$

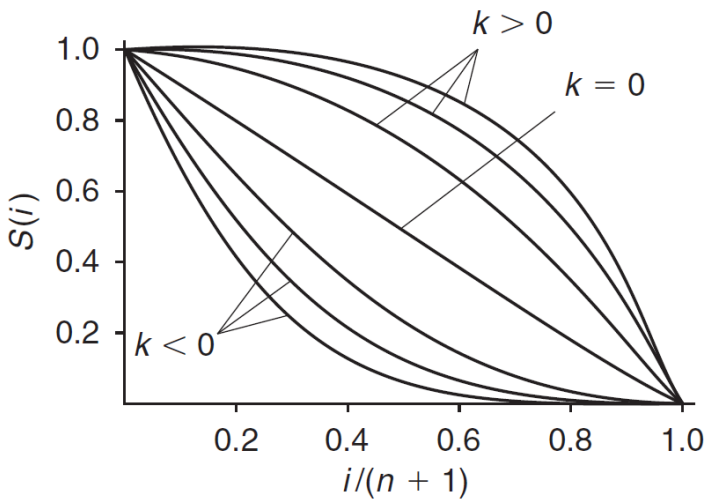
Deformation by Pulling

- Edit mesh based on a distance function
  - Euclidean
  - Shortest path along edges
  - Minimum number of edges
  - Geodesic distance on the surface
- Attenuation of the displacement

$$S(i) = 1 - \left(\frac{i}{n+1}\right)^{k+1} \qquad k \geq 0$$

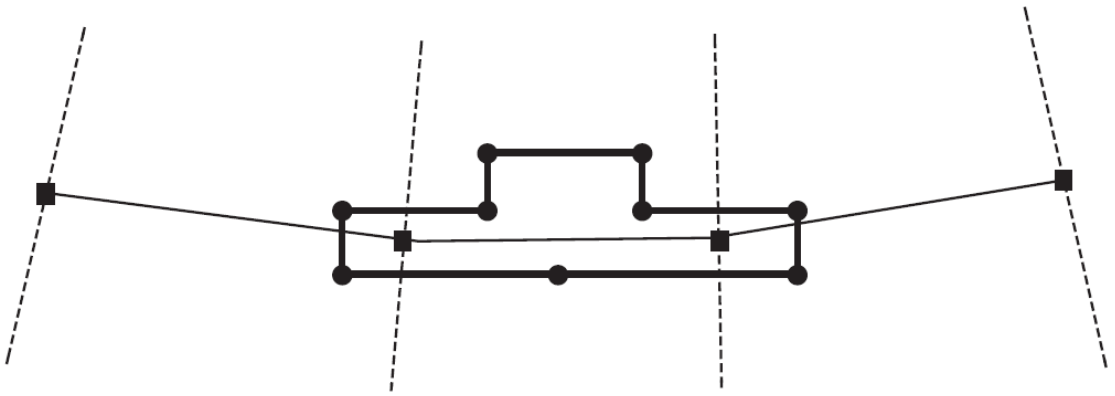
$$= \left(1 - \left(\frac{i}{n+1}\right)\right)^{-k+1} \qquad k < 0$$

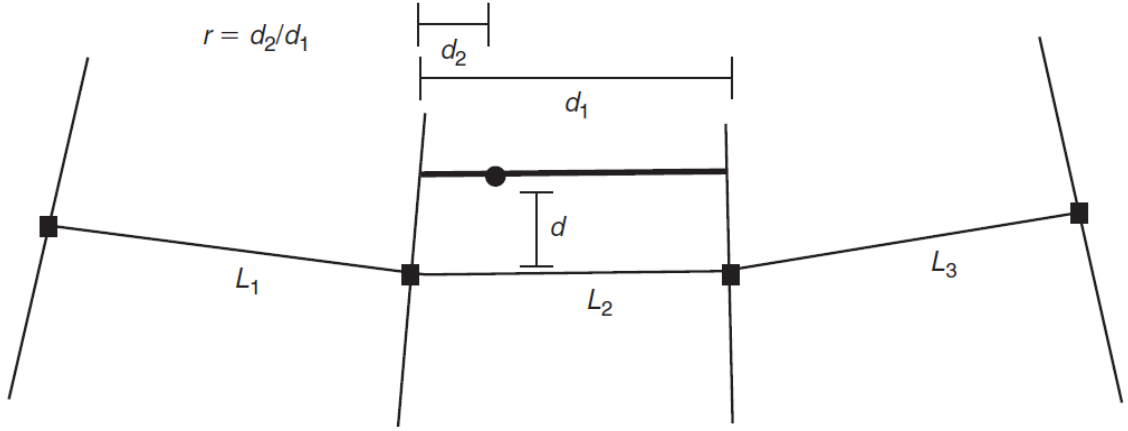


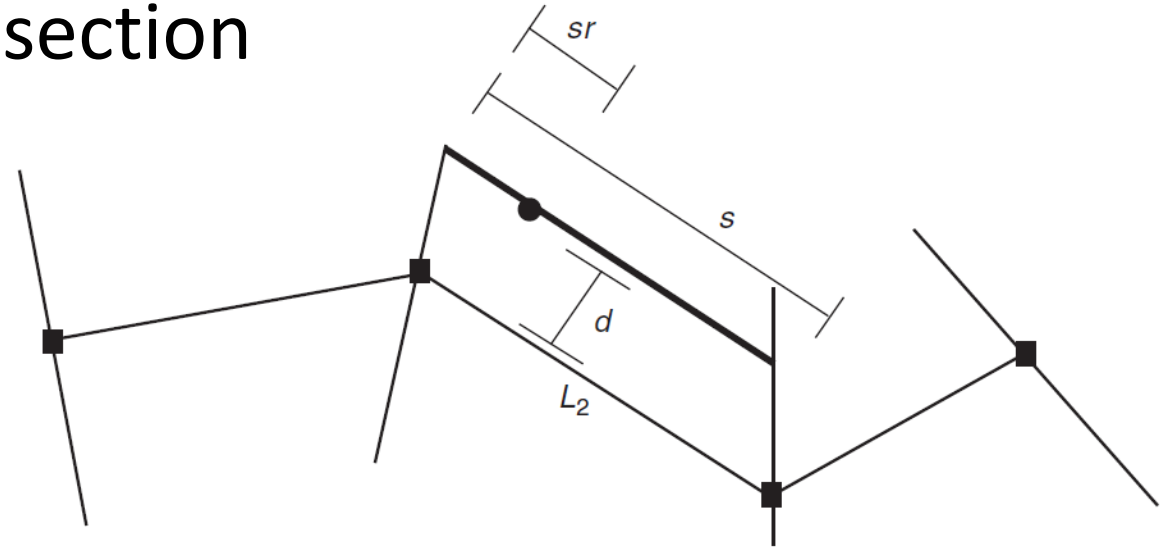


Embedding Space Deformation: Polyline

- Embedding object is a polyline defining space sections from polyline's vertex normal
- Local coordinates are distance from corresponding polyline segment and relative position within space section



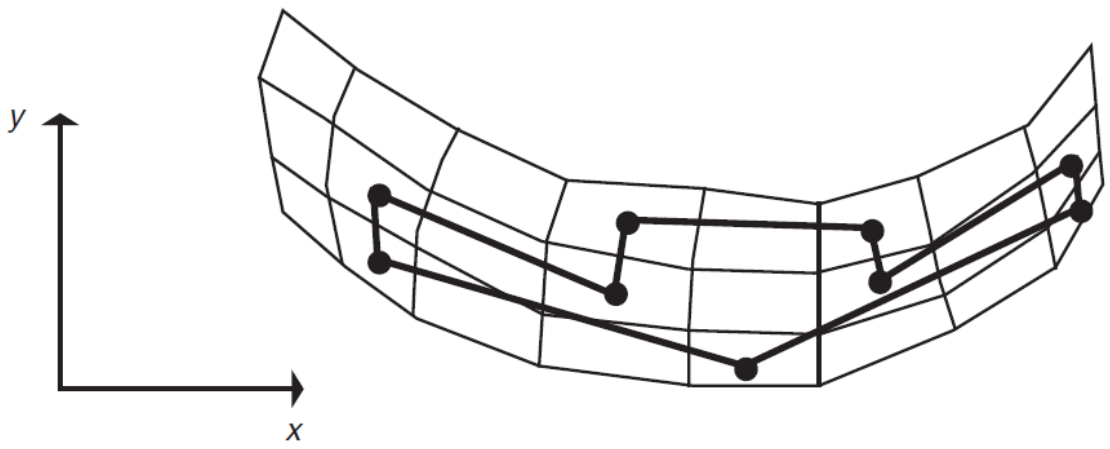


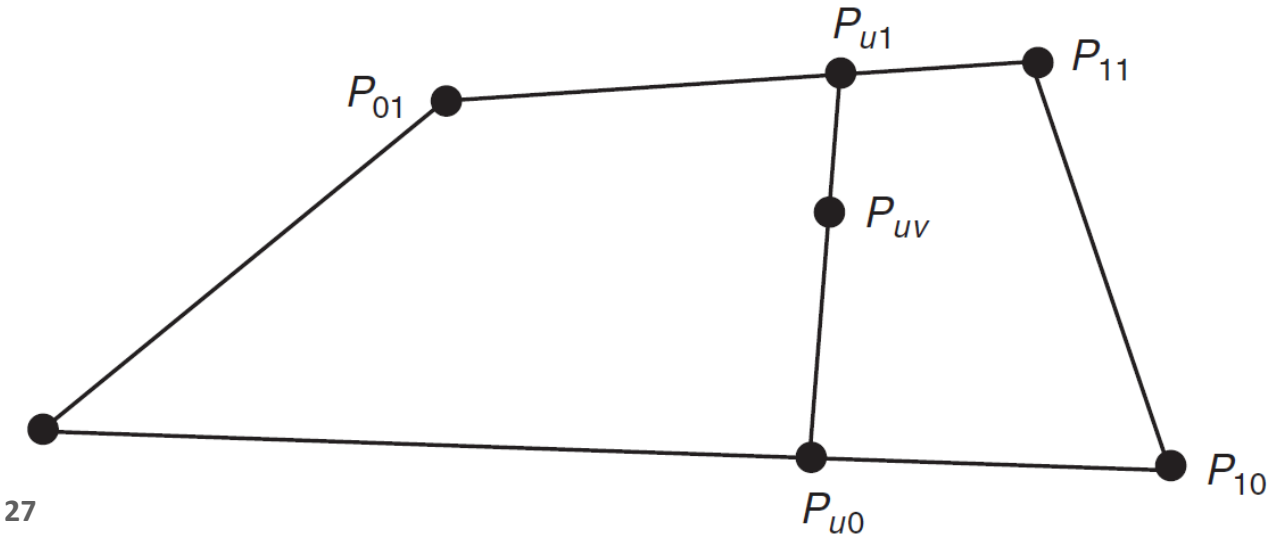


Embedding Space Deformation: Grid

- Embedding object establishes a local coordinate system with the scene object
- Deforming embedding object and preserving the scene object's vertex positions within the local coordinates, we deform the scene object itself

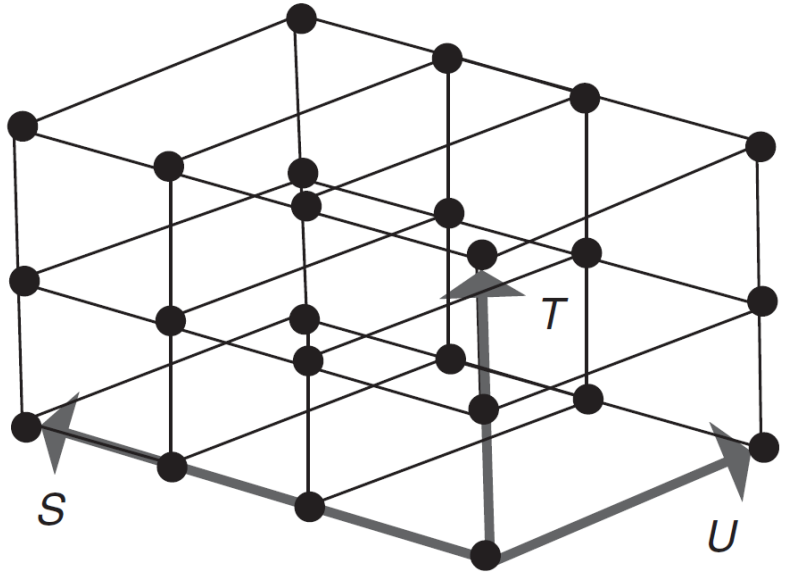
$$
\begin{aligned}
P_{u0} &= (1-u)P_{00} + uP_{10} \\
P_{u1} &= (1-u)P_{01} + uP_{11} \\
P_{uv} &= (1-v)P_{u0} + vP_{u1} \\
&= (1-u)(1-v)P_{00} + (1-u)vP_{01} + u(1-v)P_{10} + uvP_{11}
\end{aligned}
$$

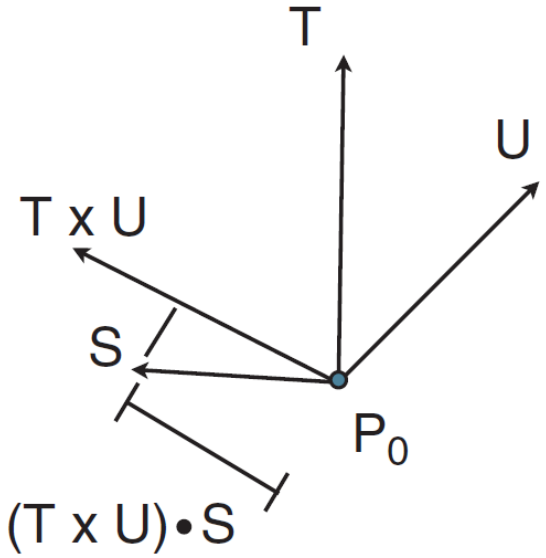


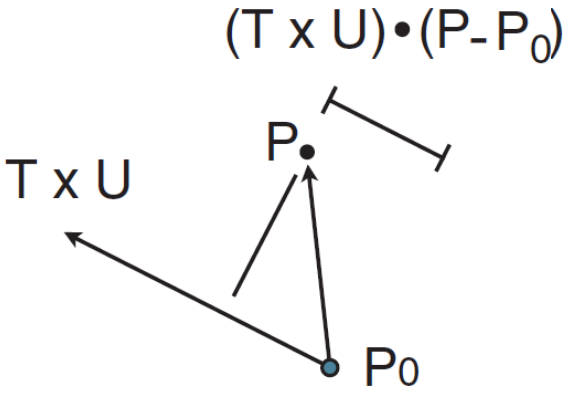


Embedding Space Deformation: Free-Form (FFD)

- Embedding object is a three-dimensional grid
- Local coord. system defined by vectors (*S*,*T*,*U*) spanning 3D space
- Point *P* is related to interpolants (*s*,*t*,*u*)







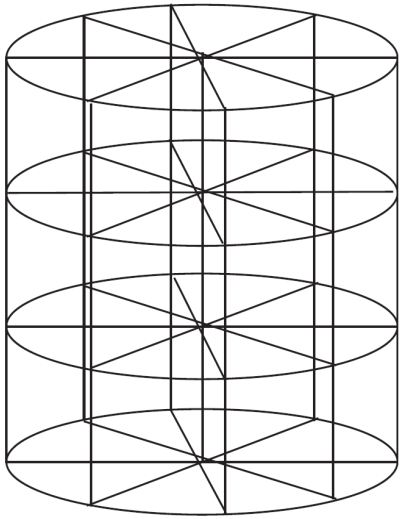
$$s = (T \times U)\cdot(P - P_0)/((T \times U)\cdot S)$$

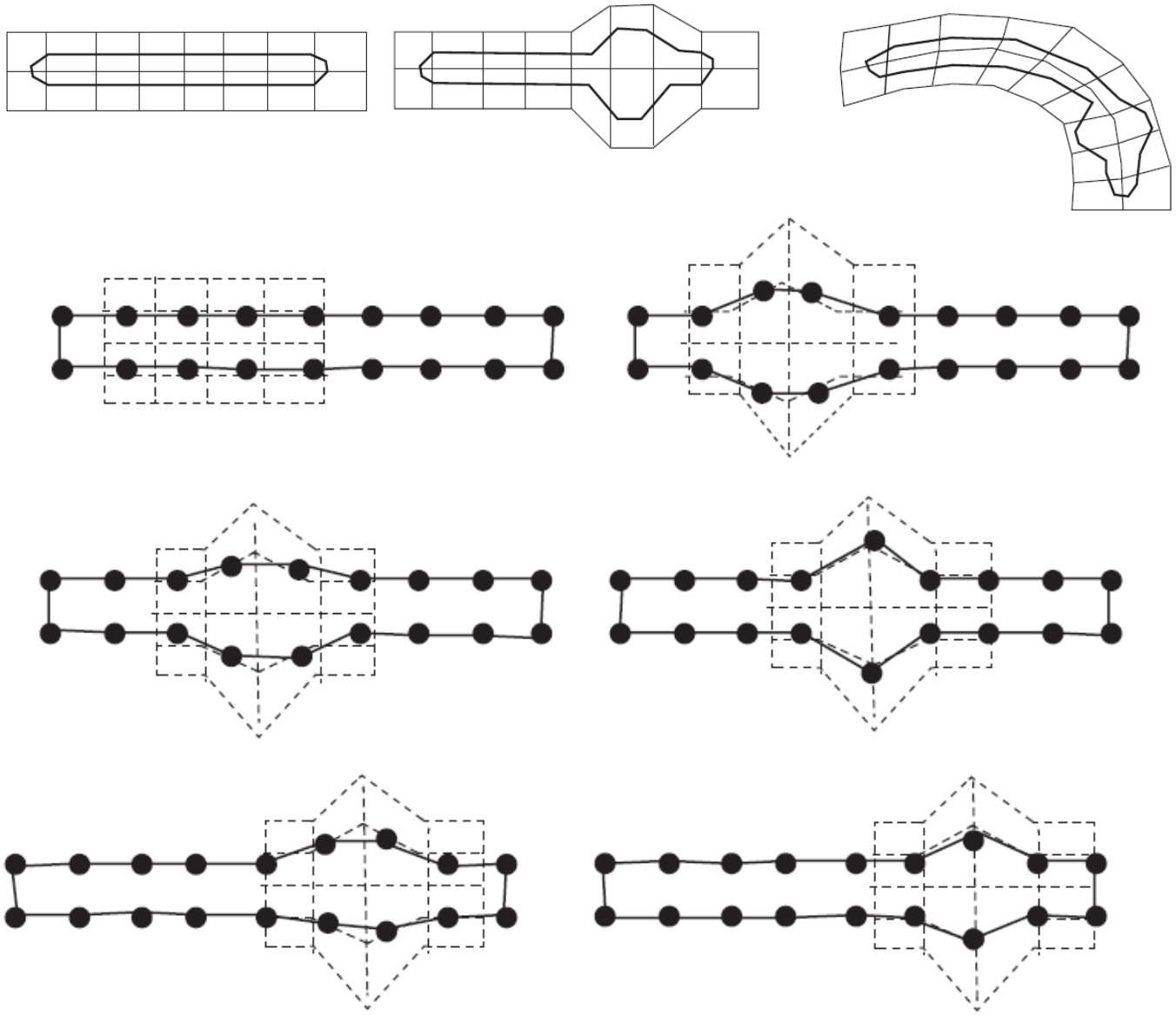
$$t = (U \times S)\cdot(P - P_0)/((U \times S)\cdot T)$$

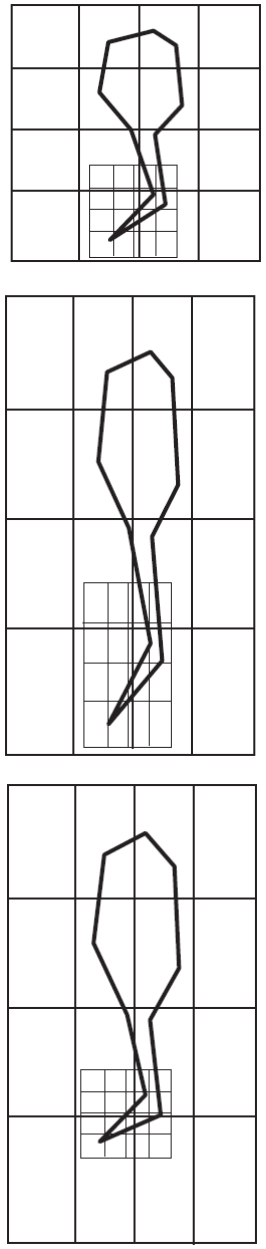
$$u = (S \times T)\cdot(P - P_0)/((S \times T)\cdot U)$$

$$P = P_0 + sS + tT + uU$$

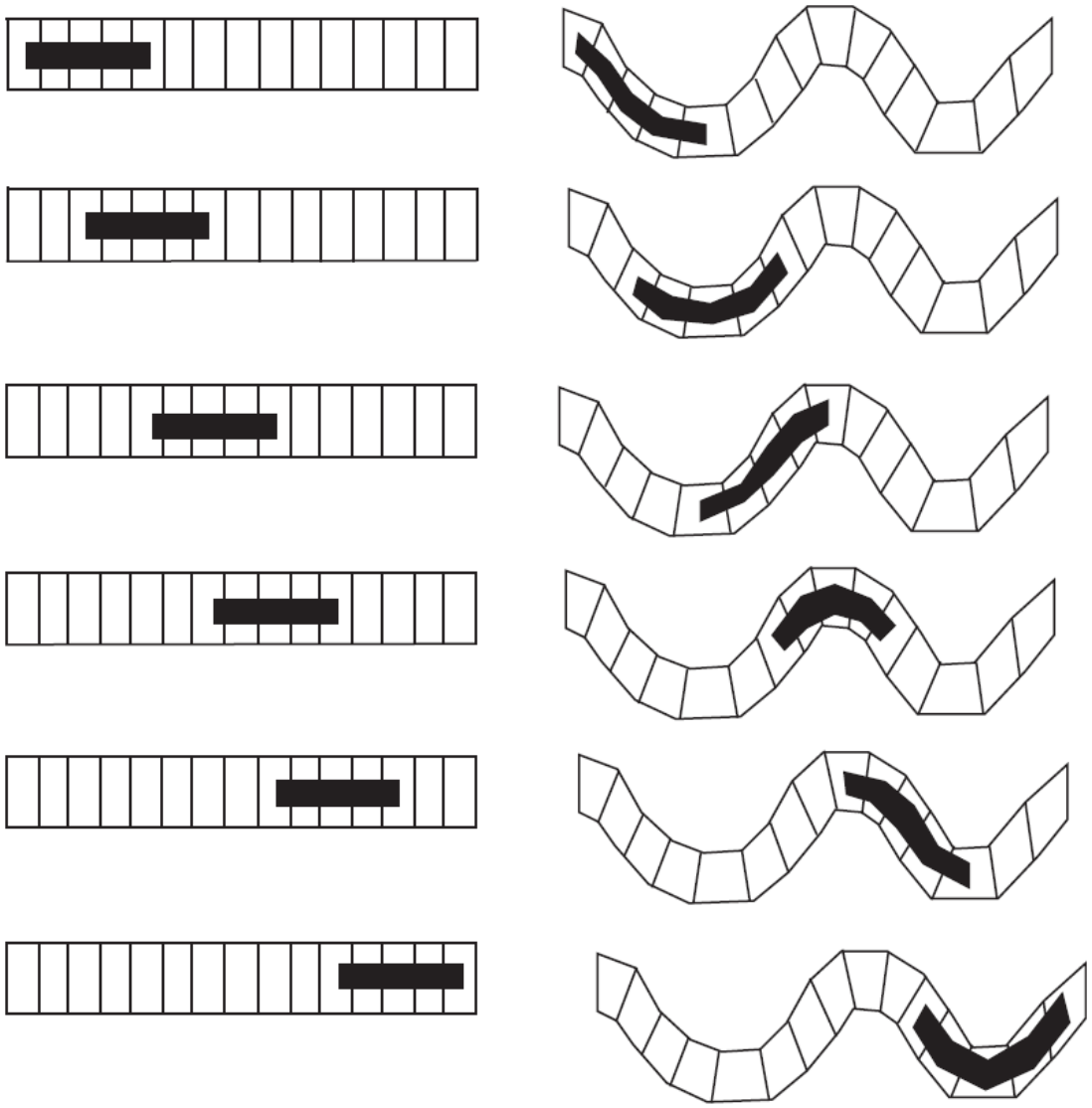
Embedding Space Deformation: Free-Form (FFD)

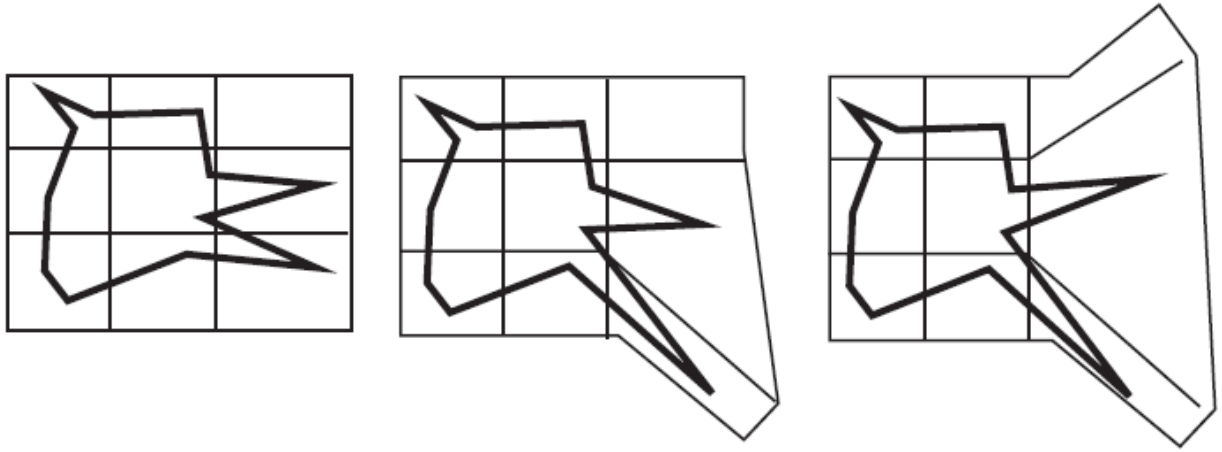


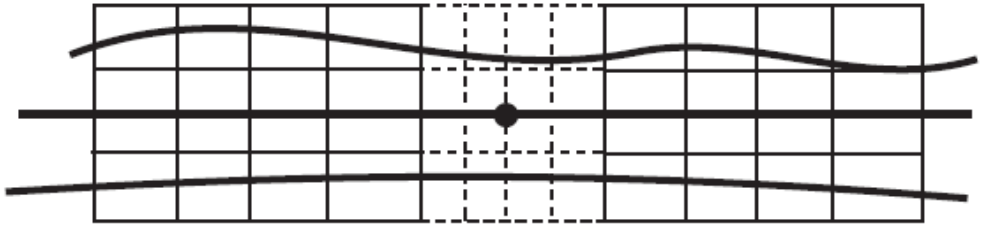




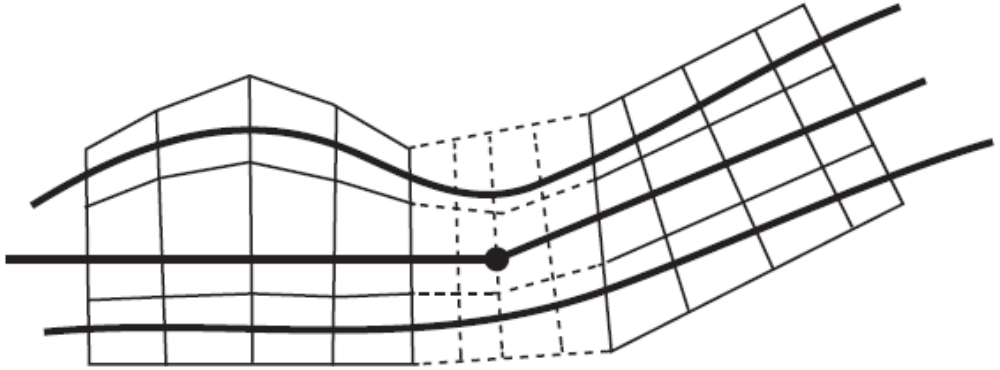
Embedding Space Deformation: Free-Form (FFD)







Initial configuration



?